

Компьютерный практикум

Тема 1. Алгоритмы и их свойства. Инструментарий Java

Преподаватель: Д. В. Давыдов

2026

Что такое алгоритм?

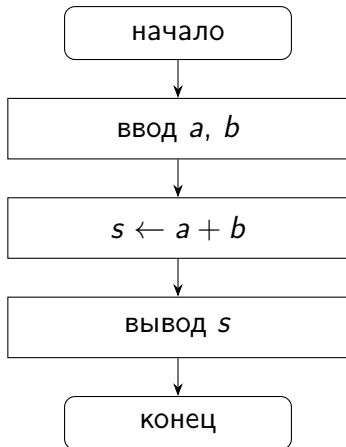
Определение (Кормен, "Алгоритмы: построение и анализ")

Алгоритм — это любая корректно определённая вычислительная процедуры, на вход которой подаётся некоторая величина или набор величин, и результатом выполнения которой является выходная величина или набор значений.

Основные виды алгоритмов

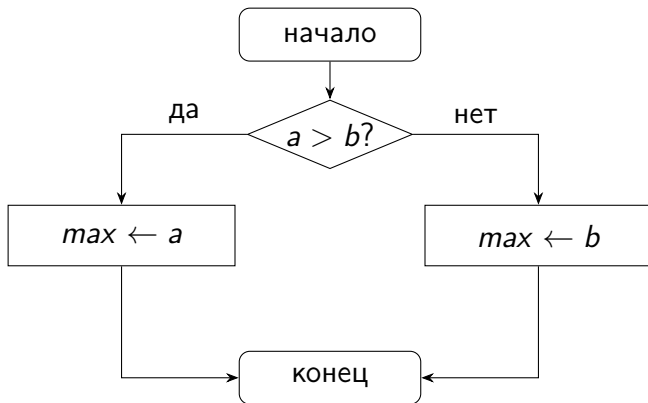
- 1 **Линейный** — действия выполняются последовательно, одно за другим;
- 2 **Разветвляющийся** — порядок действий зависит от выполнения условия;
- 3 **Циклический** — одни и те же действия повторяются несколько раз.

Линейный алгоритм



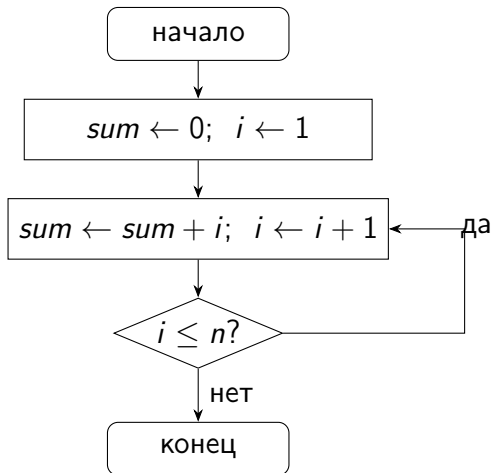
Разветвляющийся алгоритм

Пример: нахождение большего из двух чисел.



Циклический алгоритм

Пример: вычисление суммы $1 + 2 + \dots + n$.



Задания на блок-схемы / псевдокод

- Программа для вычисления площади треугольника по двум сторонам a , b и углу α между ними.

Задания на блок-схемы / псевдокод

- Программа для вычисления площади треугольника по двум сторонам a , b и углу α между ними.
- Программа для вычисления факториала заданного числа n .

Задания на блок-схемы / псевдокод

- Программа для вычисления площади треугольника по двум сторонам a , b и углу α между ними.
- Программа для вычисления факториала заданного числа n .
- По заданным коэффициентам a , b , c квадратного уравнения $ax^2 + bx + c = 0$ найти корни x_1 , x_2 , либо один корень x , либо сообщить об отсутствии корней.

Примечание

Для отрисовки блок-схем на компьютере можно воспользоваться ресурсом <https://app.diagrams.net/>

Пример: словесная запись и псевдокод

Словесное описание

Сравнить числа a и b ; если $a > b$, то наибольшим является a , иначе — b . Вывести результат.

Псевдокод

```
ввод a, b
если a > b то
    max ← a
иначе
    max ← b
вывод max
```

Пример: программа на Java

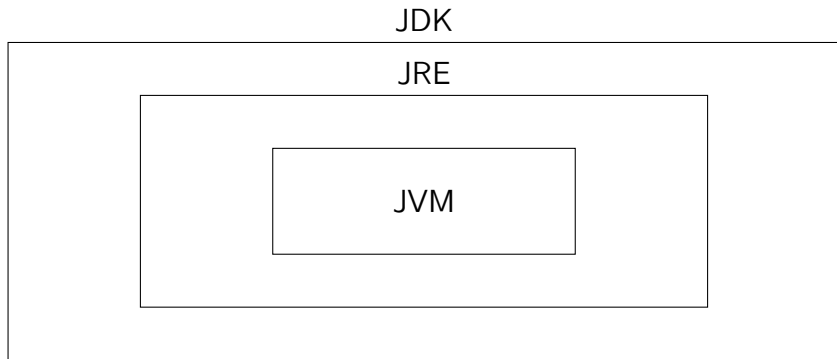
```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt();
        int b = sc.nextInt();
        int max;
        if (a > b) {
            max = a;
        } else {
            max = b;
        }
        System.out.println(max);
    }
}
```

Среда Java и её компоненты

- JVM** (Java Virtual Machine) — виртуальная машина, исполняющая байт-код;
- JRE** (Java Runtime Environment) — среда выполнения: JVM + стандартные библиотеки;
- JDK** (Java Development Kit) — комплект разработчика: JRE + компилятор `javac` + утилиты и документация.

Соотношение JDK, JRE и JVM



$JDK \supseteq JRE \supseteq JVM$: для запуска программы достаточно JRE, для разработки необходима JDK.

Написание кода и разработка

JDK разрабатывается и распространяется компанией Oracle. Скачать все инструменты разработки можно с их сайта.

<https://www.oracle.com/java/technologies/downloads/>

Для удобного написания кода используются различные редакторы кода и IDE (*integrated development environment*).

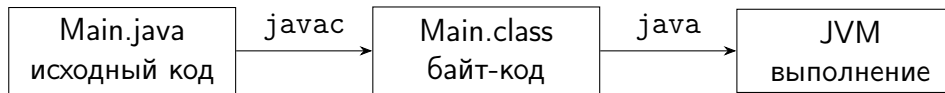
Наиболее популярные из них:

- *IntelliJIDEA* от компании JetBrains
<https://www.jetbrains.com/idea/>
- *Eclipse* <https://eclipseide.org/>
- *VSCode* и его более открытые аналоги *Code - OSS* и *VS Codium* <https://code.visualstudio.com/>

Дополнительные источники

- Тutorials от dev.java со встроенным запуском кусков кода
<https://dev.java/learn/>.
- Бесплатный онлайн курс от университета Хельсинки, вроде можно без регистрации брать материал
<https://java-programming.mooc.fi/>.
- Множество курсов и материалов в интернете.
- Документация от Oracle
<https://docs.oracle.com/en/java/javase/21/>.

От исходного кода к выполнению



- компилятор переводит исходный код в байт-код;
- байт-код выполняется виртуальной машиной JVM.

Первая программа и команды

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello, Java!");  
    }  
}
```

Команды в консоли:

```
javac Main.java  
java Main
```

Что внутри*

Инструмент *javap* позволяет дизассемблировать скомпилированный java код формата *.class* и увидеть читаемый (относительно) байткод.

```
javap -c MainHV.class
```

Что получилось:

```
Compiled from "MainHJ.java"
public class MainHJ {
    public MainHJ();
        Code:
            0: aload_0
            1: invokespecial #1                  // Method java/lang/Object."<init>":()V
            4: return

    public static void main(java.lang.String[]);
        Code:
            0: getstatic     #7                  // Field java/lang/System.out:Ljava/io/PrintStream;
            3: ldc           #13                 // String Hello, Java!
            5: invokevirtual #15                 // Method java/io/PrintStream.println:(Ljava/lang/String;)V
            8: return
}
```

Установка JDK

Разобраться с установкой JDK для своей операционной системы.
Проверить корректность установки командами в консоли:

```
java --version  
javac --version
```

GIT — система управлениями версий. Позволяет создать репозиторий и отслеживать различные состояния проекта, возвращаясь к предыдущим. Основной сайт:

<https://git-scm.com/>. Там же можно найти документацию по командам и инструкцию по установке.

GIT — система управления версиями. Позволяет создать репозиторий и отслеживать различные состояния проекта, возвращаясь к предыдущим. Основной сайт:

<https://git-scm.com/>. Там же можно найти документацию по командам и инструкцию по установке. Некоторые термины:

- Репозиторий — хранилище программного кода, изменений и сопутствующих файлов.
- Коммит — сохранённое состояние проекта.
- Ветка — отдельная линия разработки, позволяет вводить новый функционал, не нарушая работу основной программы.

Установка Git

Установить Git. Проверить установку командой:

```
git --version
```

Создать аккаунт на github <https://github.com/>. Создать новый репозиторий (на github будет подробная инструкция), добавить файл README.MD с каким-нибудь содержимым. Локально развернуть репозиторий, внести изменение в файл README.MD и залить обновление в удалённую ветку.